

Unidad 1



Instalación del sistema gestor de bases de datos

Administración de sistemas gestores de bases de datos



Índice



1.1. El administrador de bases de datos

- 11.1. Puesta en funcionamiento
- 11.2. Control y seguridad
- 11.3. Explotación y administración del SGBD

1.2. Funciones del SGBD

- 12.1. Definición de los datos: Metadatos
- 12.2. Manipulación de los datos
- 12.3. Seguridad e integridad de los datos
- 12.4. Concurrencia y transacciones
- 12.5. Auditoría y monitorización
- 12.6. Conectividad, soporte y compatibilidad
- 12.7. Independencia física y lógica del sistema

1.3. Elementos del SGBD

- 13.1. Preprocesador del lenguaje de manipulación de datos
- 13.2. Procesador de consultas
- 13.3. Compilador del lenguaje de definición de datos
- 13.4. Gestor del diccionario
- 13.5. Gestor de ficheros
- 13.6. Gestor de la base de datos
- 13.7. Interfaces externas

1.4. Clasificación de los SGBD

- 14.1. Estructura: Relacionales vs NoSQL
- 14.2. Monousuario vs Multiusuario
- 14.3. Localización: Centralizado vs distribuido
- 14.4. Tipos de licencia: propietarios vs software libre

1.5. Sistemas NoSQL

- 15.1. Ventajas de los sistemas NoSQL
- 15.2. Desventajas de los sistemas NoSQL
- 15.3. Clasificación de los sistemas NoSQL

1.6. Tipos de bases de datos

- 16.1. Bases de datos Transaccionales u operacionales
- 16.2. Datawarehouse

1.7. Características del servidor de un SGBD

- 17.1. Prioridad del servicio
- 17.2. Sistema operativo y compatibilidad
- 17.3. Hardware
- 17.4. Conectividad

1.8. Diferentes casos de uso

- 18.1. SGBD para aplicación móvil
- 18.2. Pyme
- 18.3. Tienda Online
- 18.4. Operaciones de una organización grande
- 18.5. Datawarehouse corporativo
- 18.6. Data Lake

1.9. Instalación de Oracle Database Express Edition

- 19.1. Proceso de instalación
- 19.2. Componentes instalados



Introducción

Con las competencias que vamos a adquirir durante el transcurso de este módulo, seremos capaces de desempeñar el rol de un administrador de bases de datos.

Para familiarizarnos con la nomenclatura rutinaria de un administrador de base de datos, vamos a introducir algunos de los términos más frecuentes, así como sus siglas o abreviaturas comunes que iremos detallando a lo largo de la unidad.

- > **Base de datos:** colección de información organizada de forma que se pueda acceder, administrar y mantener actualizada.
- > **SGBD Sistema Gestor de Bases de Datos (DBMS en inglés):** Software que permite el almacenamiento, modificación y extracción de los datos en una base de datos. Aporta también el conjunto de herramientas para explotar, gestionar y administrar la información contenida en las bases de datos.
- > **Consistencia:** en sistemas de base de datos se debe garantizar la coherencia de la información, para ello mediante el mecanismo de transacciones, se permite deshacer una operación o conjunto de operaciones que no se ha ejecutado de forma correcta y compacta.
- > **Integridad:** conjunto de reglas y restricciones definidas sobre los datos que garantizan la corrección, el mantenimiento y el acceso a los mismos.
- > **Transacción:** Conjunto de operaciones que solo se pueden ejecutar de manera atómica, es decir, o se ejecutan todas o ninguna.
- > **Diccionario de datos:** repositorio donde se almacenan los metadatos, es decir la información que contienen los datos (bases de datos, tablas, usuarios, etc.).
- > **NoSQL:** Tradicionalmente, los SGBD han utilizado el modelo relacional, basado en tablas (filas y columnas) relacionadas entre sí, recientemente han aparecido alternativas al modelo relacional con capacidad para procesar volúmenes muy grandes de información de forma escalable y distribuida.
- > **Administrador de base de datos (DBA):** Se denomina DBA, por sus siglas en inglés (Data base administrator) a la persona o equipo responsable de asegurar la disponibilidad de la información y el acceso de forma óptima.

Al finalizar esta unidad

- + Conoceremos los aspectos básicos relativos a los SGBD.
- + Entenderemos qué funciones básicas son las que desempeña un administrador de base de datos.
- + Exploraremos algunas de las opciones más comunes que existen en el mercado para el almacenamiento de la información mediante un SGBD matizando algunas diferencias.
- + Instalaremos un Sistema gestor de Bases de datos relacional.



1.1.

El administrador de bases de datos

Aunque el DBA tiene otras muchas responsabilidades, en esta unidad nos vamos a centrar en una de las funciones elementales que una empresa nos puede asignar, como DBA, es la instalación del SGBD, normalmente en una fase temprana de un nuevo proyecto, o también por otros motivos como realizar una migración o respaldo.

El rol del DBA, también tendrá la responsabilidad de todo el ciclo de vida del sistema de información, desde la propia elección del sistema, pasando por el mantenimiento, gestión y organización.

Podemos distinguir cuatro grupos de funciones que un DBA debe ser capaz de desempeñar dentro de una organización.

1.1.1. Puesta en funcionamiento

Elección: deberemos decidir qué gestor de los que hay disponibles en el mercado se adapta mejor al proyecto, así como al presupuesto de la empresa o departamento. Para ello, debemos conocer bien las alternativas, y especialmente, decantarnos por un **sistema libre (p. e. MariaDB, PostgreSQL) o propietario (Oracle, SQL Server, ...)**.

Despliegue, Instalación y configuración del SGBD: Esta decisión dependerá de la infraestructura de servidores de la organización. Disponemos de alternativas con soluciones completas en la nube, que no requieren instalación del tipo Software como Servicio (SaaS), instalación directa sobre un servidor físico (más tradicionales) o soluciones híbridas mediante servidores genéricos a través de contenedores o máquinas virtuales e incluso clúster de servidores.

En organizaciones de tamaño grande, el equipo de DBAs, va a asesorar al equipo de sistemas durante la instalación del SGBD, en cuanto a dimensionamiento y configuración.

1.1.2. Control y seguridad

Gestión de usuarios y permisos: tendremos que saber crear usuarios y establecer para cada los niveles de acceso (permisos), tanto en la fase de configuración inicial como durante la explotación.

Seguridad: realizaremos auditorías de seguridad de forma periódica. Implementaremos mecanismos que puedan detectar intentos de intrusión, anomalías, conforme a las políticas de seguridad establecidas por la organización. Obviamente, este aspecto es fundamental, como DBA, debemos estar preparados para la recuperación de los datos y del servicio en caso de ataque o pérdida de datos. Esto incluye la realización periódica de copias de seguridad.

Privacidad y protección de los datos: debemos garantizar el cumplimiento de las leyes vigentes, así como las medidas de seguridad y privacidad internas establecidas en el plan de seguridad de la empresa.

1.1.3. Explotación y administración del SGBD

Adaptación: tendremos que asegurar la disponibilidad del servicio, así como un funcionamiento óptimo del mismo, garantizando la escalabilidad del sistema. Dependiendo de la evolución y del crecimiento de los datos de la organización, puede suponer cambiar la arquitectura, realizar migraciones a equipos más potentes o incluso modificar parcial o completamente el gestor de bases de datos.

Colaboración: nuestra labor como administradores de bases de datos es estar en permanente comunicación, con el equipo de desarrollo y el resto de los administradores de sistemas.



1.2.

Funciones del SGBD

Los sistemas gestores de bases de datos han aparecido para dar solución a los problemas de almacenamiento de datos tradicional, basado en archivos. Esos problemas son la falta de consistencia, disponibilidad, integridad y durabilidad de los datos, añadiendo funcionalidades adicionales.

Las principales **funciones** que debe proporcionar un SGBD son: **Definición** y **manipulación** de datos, aportar **seguridad** e **integridad** en los datos, gestión de acceso **concurrente**, gestión de **transacciones**, **auditoría**, **tiempo de respuesta máximo garantizado** e **independencia física y lógica**.

1.2.1. Definición de los datos: Metadatos

Por medio de los **metadatos** almacenados en el **diccionario de datos**, vamos a definir nuestra estructura de datos, esto es, esquemas, bases de datos, jerarquía, tablas, así como su descripción de todos los objetos. El contenido y formato de los metadatos es dependiente del propio SGBD, su arquitectura y jerarquía.

Se definen a través del lenguaje de definición de datos (**DDL**), compuesto por una serie de comandos para crear y/o modificar los objetos.

1.2.2. Manipulación de los datos

A través del lenguaje de manipulación de datos (**DML**) podremos realizar las cuatro funciones fundamentales conocidas habitualmente como **CRUD**: *Create* (**crear**), *Read* (**leer**), *Update* (**actualizar**) y *Delete* (**eliminar**).

Cada SGBD tiene sus propios DDL y DML. En modelos relacionales, el más popular es el lenguaje declarativo SQL, sin embargo, en modelos jerárquicos, el programador debe especificar mediante lenguajes procedimentales la ruta de acceso a los datos.

1.2.3. Seguridad e integridad de los datos

Debemos utilizar los mecanismos de los que cada SGBD disponga para garantizar la seguridad e integridad de los datos. Esto incluye autenticación de usuarios, permisos, encriptación de los datos, protección ante fallos físicos (discos redundantes y copias de seguridad), monitorización y auditoría.

1.2.4. Concurrencia y transacciones

Para garantizar la concurrencia, nuestro SGBD debe ser capaz de soportar el acceso a los datos de forma simultánea por un gran número de usuarios simultáneamente. Para ello, durante la ejecución de un bloque transaccional, se restringen temporalmente ciertas operaciones que puedan incumplir la condición de concurrencia.

1.2.5. Auditoría y monitorización

Nuestro SGBD dispondrá de herramientas que permitan analizar su funcionamiento, así como la actividad de los diferentes usuarios (accesos de usuarios, tablas bloqueadas, pérdidas de conectividad, paradas del servicio, etc.).

1.2.6. Conectividad, soporte y compatibilidad

En el momento de tomar una decisión sobre la elección de nuestro SGBD, debemos valorar que sea un sistema con una buena comunidad de usuarios (en caso de sistema libre) o una calidad de soporte (si el sistema es de pago), además de disponer libremente de múltiples conectores y librerías compatibles con las principales plataformas de datos y lenguajes de programación.

1.2.7. Independencia física y lógica del sistema

El SGBD debe garantizar una latencia máxima con independencia del sistema donde se encuentre (SaaS, cloud, servidor local, contenedor o sistema distribuido). Para el usuario, el acceso debe ser totalmente transparente, es decir, el usuario se autentica y opera sobre las bases de datos con independencia de su ubicación física y arquitectura lógica. Debemos ser cautelosos con la arquitectura, especialmente en sistemas con requisitos de muy baja latencia.



1.3.

Elementos del SGBD

Los componentes del SGBD pueden variar mucho de unos sistemas a otros, aunque tienen elementos comunes, ya que sus funciones son similares, debemos conocer las características de esas diferencias.

1.3.1 Preprocesador del lenguaje de manipulación de datos

El preprocesador del DML es una herramienta que se utiliza para modificar el código fuente de un programa antes de que éste se compile. El preprocesador del DML se utiliza principalmente para añadir funcionalidad a un programa o para corregir errores en el código fuente.

1.3.2 Procesador de consultas

Tiene la función de transformar las consultas en el lenguaje de consultas, habitualmente SQL en SGBD relacionales, a las bases de datos a instrucciones de bajo nivel que ejecutará el SGBD.

El procesador de consultas de las bases de datos relacionales entiende los tipos de componentes del lenguaje SQL:

- > **DDL:** Lenguaje de definición de los datos. Las instrucciones habituales son: CREATE, ALTER, DROP, GRANT, TRUNCATE, etc.
- > **DML:** Lenguaje de manipulación de datos. Las instrucciones que usaremos con más frecuencia son las CRUD: INSERT, SELECT, UPDATE y DELETE.
- > **TCL:** Lenguaje de control de transacciones: COMMIT, ROLLBACK SET TRANSACTION, SET CONSTRAINT, etc. Algunos sistemas lo integran en el DML, pero considerarán las instrucciones como un conjunto denominado sentencias de control de transacciones.
- > **DCL:** Lenguaje de control de datos. Nos va a permitir definir los privilegios de los usuarios sobre los datos a través de sentencias SQL como GRANT y REVOKE.
- > **SCL:** Lenguaje de control de sesión: Podremos modificar las características de la sesión, así como activar y desactivar los privilegios de un usuario
- > **SQL Embebido:** Algunos lenguajes procedimentales integran las sentencias SQL en su compilador.

1.3.3. Compilador del lenguaje de definición de datos

Su función es transformar las sentencias del DDL en un conjunto de tablas que contienen los metadatos y enviarlas al diccionario de datos, que almacena los esquemas de las diferentes bases de datos del SGBD.

1.3.4. Gestor del diccionario

El diccionario contiene una base de datos con la información sobre el resto de los datos, varía de unos SGBD a otros. Con los permisos apropiados, podremos escribir sentencias SQL para obtener información sobre la estructura de datos. Lo veremos en más detalle más adelante.

En caso de sistemas de gestión distribuidos (SGBDD), es decir que funcionan sobre un clúster de máquinas, en lugar de una solamente, la complejidad de arquitectura se eleva, debido a que el sistema tiene que resolver la casuística de comunicación, y de estructura interna del sistema distribuido, es decir, en qué nodo del clúster se encuentra la información.

1.3.5. Gestor de ficheros

Gestiona el acceso físico a los ficheros en disco donde se guarda la información. Este proceso lo realiza a través del sistema operativo, no de forma directa.



1.3.6. Gestor de la base de datos

Es el elemento más importante. Recibe y procesa todas las instrucciones de los usuarios. Cuando recibe una sentencia, consulta el diccionario de datos para determinar dónde están los recursos y pasar la petición al gestor de ficheros que devolverá la información. Internamente está organizado en los siguientes módulos:

- > **Control de autorización:** Verifica que el usuario puede ejecutar la operación que ha solicitado.
- > **Procesador de comandos:** Interpreta las sentencias enviadas por el usuario.
- > **Control de la integridad:** Cuando una operación realiza modificaciones sobre los datos, comprueba que se cumplen las reglas de integridad previamente definidas (claves, tipos de datos, etc.)
- > **Optimizador de consultas:** Reduce el tiempo en procesar las consultas. No excluye al administrador y al usuario del uso de las buenas prácticas tanto en diseño de la estructura como de las propias sentencias SQL.
- > **Gestor de transacciones:** Es la parte del software del SGBD que implementa y garantiza la atomicidad en la ejecución de transacciones.
- > **Planificador:** Asegura la concurrencia, evitando conflictos debidos al acceso simultáneo a los datos.
- > **Gestor de recuperación:** Guarda estados para garantizar la consistencia de los datos para poder volver al punto anterior al fallo sin pérdida de información ni coherencia de los datos.
- > **Gestor de buffers:** Se encarga de repartir los recursos de memoria de acceso rápido y discos.

1.3.7. Interfaces externas

A través de las interfaces externas, nuestros SGBD permiten a los usuarios y aplicaciones interactuar con ellos. Son implementadas a través de conectores que proporcionan una interfaz de acceso, tanto para entornos de programación como para programas que integran diferentes fuentes de datos, entre ellas las que provienen de SGBD.

ODBC es una interfaz de programación de aplicaciones (API) de software que proporciona una forma estándar de acceso a la base de datos. ODBC hace que sea posible acceder a datos almacenados en una base de datos utilizando una variedad de lenguajes de programación y herramientas que requieren acceso a los datos. También es muy popular el uso de JDBC, que es válida únicamente para java.

Incluiremos en esta categoría las interfaces que permiten modelar la base de datos a través de un lenguaje de programación orientado a objetos como Hibernate, Entity Framework, aunque son dependientes del entorno de programación, pero independizan el desarrollo a alto nivel del SGBD.



1.4.

Clasificación de los SGBD

Los factores determinantes para elegir un DBMS dependen del **tamaño** y la **complejidad** de la organización. Algunos de los factores clave a considerar incluyen el **costo**, el **rendimiento**, la **escalabilidad**, la **fiabilidad**, la **seguridad** y el **soporte técnico**. A continuación, vamos a realizar una clasificación según diferentes criterios.

1.4.1. Estructura: Relacionales vs NoSQL

Los sistemas gestores de bases de datos relacionales se basan en el modelo relacional de datos, que establece una estructura de datos tabular con un conjunto de registros que están relacionados entre sí. Los RDBMS permiten a los usuarios realizar consultas y actualizaciones complejas a través del uso de lenguajes de consulta estructurados, como SQL.

Los más importantes de esta categoría son Oracle, SQL Server, PostgreSQL, Access o MySQL (MariaDB en su implementación alternativa abierta y gratuita de MySQL). Con el aumento significativo de la utilización de sistemas en la nube, como DBA, debemos estar al tanto de los nuevos sistemas de bases de datos relacionales implementadas de forma native sobre entornos cloud como AWS (Amazon RDS). Para datos estructurados de gran volumen, es recomendable valorar soluciones como BigQuery de Google, compatible con lenguaje SQL.

Los sistemas gestores de bases de datos no relacionales (NoSQL) se basan en una variedad de modelos de datos, incluyendo columnas, claves-valor, grafos y documentos.

NoSQL ofrece un rendimiento y escalabilidad superiores a RDBMS en entornos de alta concurrencia, pero a menudo carecen de la funcionalidad de consulta y actualización compleja de RDBMS.

Es muy frecuente que ciertas organizaciones decidan realizar una migración total o parcial hacia NoSQL, por motivos diversos, sobre todo, por rendimiento y coste computacional. En cualquier caso, el factor fundamental es la naturaleza estructurada de los datos con los que la organización trabaje.

1.4.2. Monousuario vs Multiusuario

Los SGBD monousuario utilizan un modelo de acceso exclusivo, en el que solo un usuario puede acceder a la base de datos a la vez. Esto hace que los RDBMS monousuario sean más rápidos y eficientes, ya que no hay contención de recursos. Sin embargo, el modelo de acceso exclusivo también significa que los RDBMS monousuario son menos escalables y pueden no ser adecuados para entornos de alta concurrencia.

Los RDBMS multiusuario utilizan un modelo de acceso concurrente, en el que varios usuarios pueden acceder a la base de datos al mismo tiempo. Esto hace que los RDBMS multiusuario sean más escalables, ya que pueden manejar un mayor número de usuarios simultáneos.

Un ejemplo de un sistema DBMS monousuario es Microsoft Access, mientras que un ejemplo de un sistema DBMS multiusuario es Microsoft SQL Server.



1.4.3. Localización: Centralizado vs distribuido

Los sistemas DBMS centralizados utilizan una única base de datos que está alojada en un servidor central. Los sistemas DBMS centralizados son más fáciles de administrar y mantener, ya que todos los datos se encuentran en un único lugar. Sin embargo, los sistemas DBMS centralizados son menos escalables y pueden ser más propensos a fallos si el servidor central se cae.

Los sistemas DBMS distribuidos utilizan una base de datos distribuida que está alojada en varios servidores. Los sistemas DBMS distribuidos son más escalables y pueden manejar un mayor número de usuarios simultáneos. Sin embargo, los sistemas DBMS distribuidos son más difíciles de administrar y mantener, ya que los datos están dispersos en varios servidores.

1.4.4. Tipos de licencia: propietarios vs software libre

En el momento de elegir el SGBD, este aspecto es fundamental, podemos decantarnos por un sistema propietario, con licencia por uso, o sistemas de código abierto. Veamos las particularidades de cada tipo.

SGBDR propietarios

- > **Calidad del soporte técnico.**
- > **Optimizados** para un determinado SO y/o lenguaje de programación.
- > **Interfaz** de mejor calidad.

Algunos de los sistemas relacionales propietarios más utilizados son:

- > **Oracle SQL:** Es uno de los gestores de bases de datos relacionales más potentes, soporta también otro tipo de paradigmas para almacenar objetos y NoSQL. Como desventaja, tenemos su coste.
- > **Microsoft SQL Server:** Disponible para sistemas operativos Windows. Aunque es posible realizar la instalación sobre Linux, no se recomienda.
- > **Db2:** Propiedad de IBM, destaca la integración de archivos XML, que permite realizar búsquedas relacionales entre ellos.
- > **Microsoft Access:** Base de datos de la suite Office, más orientada a usuarios con escasos conocimientos de bases de datos. Permite organizar la información.

SGBDR de software libre

- > **Licencia gratuita.**
- > Pueden ser modificados, pues su **código fuente es accesible**.
- > **Comunidad de usuarios extensa.** Aunque no es garantía, en la mayoría de las ocasiones, encontraremos las soluciones a dudas o problemas que puedan surgir a través de foros, webs específicas, etc.

Dentro de esta categoría, cabe mencionar:

- > **MariaDB:** Cuando Oracle adquirió MySQL, la comunidad de desarrolladores de software libre creó MariaDB a partir de MySQL.
- > **PostgreSQL:** Sigue los estándares de SQL con más fidelidad que MariaDB/MySQL. Para un volumen importante de datos estructurados, funciona mejor que MariaDB. En los últimos años, ha ganado popularidad respecto a MariaDB. Tiene su propio lenguaje procedimental, muy parecido al de Oracle.
- > **SQLite:** es una librería que implementa un motor de base de datos SQLite. Está escrito en lenguaje C y se distribuye bajo la licencia BSD. Es autocontenido, lo que significa que no requiere ninguna configuración ni administrador de base de datos. Se ejecuta en la mayoría de los dispositivos móviles y está incluido en muchas aplicaciones de software.



1.5.

Sistemas NoSQL

Ya hemos visto en módulos anteriores los problemas que tienen las bases de datos relacionales.

Uno de ellos es su falta de **escalabilidad**, lo cual, con frecuencia es un problema debido a la velocidad de crecimiento, tanto horizontal (falta de **flexibilidad** de representación de la información) como vertical (límite del volumen de información de una organización), por lo que no son apropiados para grandes aplicaciones de internet del tipo redes sociales, buscadores, servicios de streaming, y en general para sistemas o aplicaciones que podríamos clasificar como big data.

1.5.1. Ventajas de los sistemas NoSQL

Para solucionar los problemas mencionados característicos de los SGBD relacionales, los sistemas NoSQL recurren a las siguientes estrategias:

- > **Información y procesamiento distribuido:** La información puede distribuirse en múltiples nodos, de este modo, el sistema es escalable, incrementando el número de nodos.
- > **Manejo de datos no estructurados:** No es necesario fijar una estructura para los metadatos, por lo tanto, la información se guarda en estructuras más cercanas al contexto real. Esto evita tener que enlazar múltiples tablas, mejorando considerablemente la velocidad de acceso a los datos, aunque tengamos información redundante.

1.5.2. Desventajas de los sistemas NoSQL

Al no poder establecer relaciones, es más difícil extraer información, especialmente realizar búsquedas avanzadas combinando diferentes conjuntos de datos.

Para poder realizar un análisis en profundidad de los datos, muchas veces se requiere tener almacenada en bruto la información en un SGBD de tipo no estructurado, y posteriormente estructurarlos (parcialmente) para poder explotarlos mediante un SGBD relacional.

Normalmente, la tarea de estructuración de datos no suele ser una tarea del DBA, pero suele apoyar técnicamente a los equipos de Business Intelligence (Analistas, Ingenieros de datos, etc.).

PARA TENER EN CUENTA...

El nombre NoSQL no significa que no use SQL. Viene de las siglas "Not Only SQL", por lo que podría existir un SGBD que soporte lenguaje SQL, aunque por su estructura haya sido clasificado como "NoSQL". Un ejemplo de ello es BigQuery, que soporta sintaxis SQL, pero no cumple con todas las características de un sistema relacional.

MongoDB, Redis, CouchDB y Cassandra son algunos ejemplos de sistemas de base de datos NoSQL. También a tener en cuenta los sistemas cloud nativos como Amazon DynamoDB, Azure Cosmos DB o cloud BigTable de Google.



1.5.3. Clasificación de los sistemas NoSQL

Clave-valor (key-value)

Una base de datos NoSQL key-value es una base de datos que almacena datos como una colección de pares clave-valor. Las claves se utilizan para identificar los datos y los valores pueden contener cualquier tipo de datos.

Algunos ejemplos son: Redis, Amazon DynamoDB, Azure Cosmos DB, Memcached

Columnares

Una base de datos NoSQL columnar es una base de datos que almacena datos como una colección de columnas. Las columnas se pueden agrupar en familias y se pueden almacenar en cualquier orden.

Podemos encontrar: Apache Cassandra, Google Cloud BigTable, Hbase y Vertica

Documentales

Una base de datos NoSQL de tipo documentos es una base de datos que almacena datos como documentos. Los documentos pueden contener cualquier tipo de datos, incluyendo texto, imágenes, videos, etc.

MongoDB es de las más populares, pero podríamos mencionar también Elasticsearch, Apache CouchDB, Amazon SimpleDB.

Grafos

Una base de datos NoSQL de tipo grafos es una base de datos que almacena datos como grafos. Los grafos están compuestos por nodos y aristas, donde los nodos representan los datos y las aristas representan las relaciones entre los datos.

La más popular en este caso es Neo4j. Cabe mencionar también Amazon Neptune.

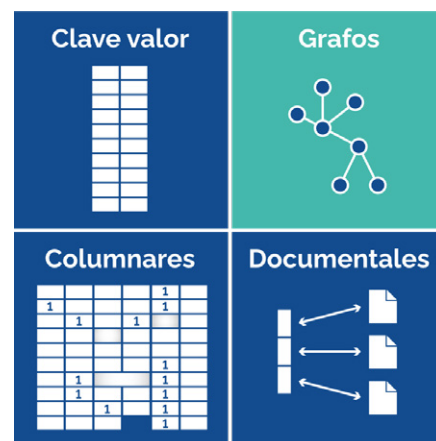


Imagen 1. Tipos del almacenamiento de datos en estructuras NoSQL

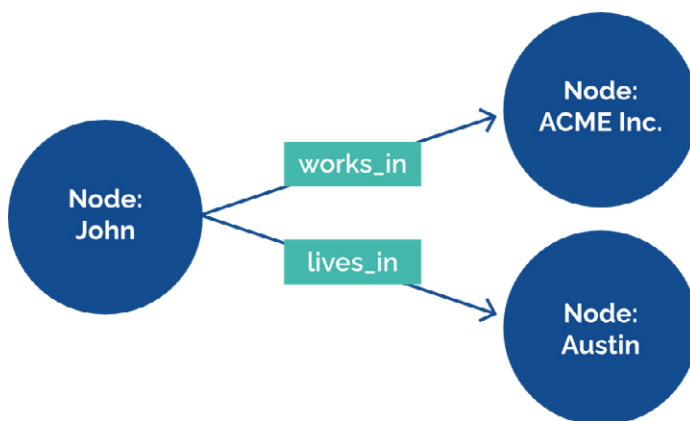


Imagen 2. Ejemplo de grafo NoSQL



1.6.

Tipos de bases de datos

A la hora de decantarnos por un SGBD, es importante tener en cuenta el tipo de acceso que vamos a necesitar y el tiempo máximo que podemos permitirnos tener las tablas bloqueadas. En base a este criterio, podemos clasificar las bases de datos en **transaccionales y datawarehouse**.

1.6.1. Bases de datos Transaccionales u operacionales

Las bases de datos transaccionales se utilizan para almacenar datos que deben mantenerse consistentes en todo momento. Por ejemplo, si una aplicación está realizando una transacción que involucra a varios usuarios, es importante que los datos de cada usuario se mantengan consistentes. De lo contrario, podría haber errores o datos corruptos. Se suelen asociar con herramientas organizativas de empresas CRM o ERP.

1.6.2. Datawarehouse

Un data warehouse es una base de datos optimizada para el almacenamiento de datos históricos. Se utiliza para el análisis de datos y la toma de decisiones. Los data warehouses suelen tener una gran cantidad de datos, por lo que se almacenan en servidores de alto rendimiento.





1.7.

Características del servidor de un SGBD

En el momento de la implantación y configuración de un SGBD, nos enfrentaremos a la toma de decisiones en múltiples parámetros en cuanto al servidor se refiere. La casuística puede ser muy variada y debemos valorar todos los escenarios cuando vayamos a realizar el dimensionamiento.

A continuación, detallaremos algunos de esos requisitos tanto de software como de hardware.

1.7.1. Prioridad del servicio

Dependiendo de las necesidades del proyecto, presupuesto, políticas de seguridad, arquitectura IT de la organización, nos enfrentaremos a varios escenarios en cuanto a las características de utilización del servidor.

Compartido

La infraestructura interna dispone o plantea instalación de un servidor común para varios servicios de red de la organización (servidor web, servidor de SGBD, otros servicios de red, etc.).

Dedicado

En el otro escenario, utilizaremos una máquina, ya sea física o virtual, local o en la nube en exclusividad para los servicios del SGBD. En el primer escenario, tendremos que evaluar los recursos que vamos a poder dedicar al servicio para no saturar el equipo ni bajar el rendimiento del resto de servicios.

Tanto en un escenario como en otro, debemos valorar, si es posible la utilización de tecnologías de contenedores, que nos van a aislar la instalación servicio del resto de servicios. Es un proceso parecido al de la virtualización, pero más optimizado que una máquina virtual tradicional a nivel de hardware/software, pues los núcleos de los procesadores actuales lo soportan de forma nativa.

El uso de contenedores tiene ciertas ventajas, pues es posible empaquetar el SO y el SGBD en una única instalación, además evitaremos fallos críticos debidos a picos de consumos de recursos simultáneos entre diferentes servicios. Como desventajas, estaremos reservando espacio en disco y memoria en exclusividad que no siempre vamos a estar utilizando.

SaaS

Aunque suele ser un producto más caro, la mayoría de los proveedores cloud, ofrecen servicios de bases de datos más populares, tanto relacionales como NoSQL, para que los administradores no tengan que preocuparse de la instalación, configuración y mantenimiento del software gestor de bases de datos. A pesar del alto coste del servicio, la organización se va a ahorrar gran parte de los costos de administración. Además, facilitan enormemente la escalabilidad y las migraciones, en caso necesario.

1.7.2. Sistema operativo y compatibilidad

El sistema operativo afecta la elección del DBMS en el sentido de que algunos DBMS se ejecutan mejor en determinados sistemas operativos que en otros. Por ejemplo, Oracle se ejecuta bien en plataformas UNIX, mientras que Microsoft SQL Server se ejecuta mejor en plataformas Windows. Debemos tener en cuenta también que el sistema operativo tenga soporte.

Normalmente los desarrolladores informan de la fecha máxima de soporte para la versión actual del sistema operativo.

No debemos pasar por alto la madurez de las librerías o paquetes necesarios que aportan conectividad entre plataformas.

1.7.3. Hardware

Los requisitos de hardware van a depender mucho del escenario disponible en cuanto a las características del servicio mencionadas anteriormente.

Disco duro

Los SGBD son grandes consumidores de este recurso, por lo que es necesario dimensionarlo apropiadamente, y disponer de algún mecanismo que permita la escalabilidad. Oracle requiere 8 GB de disco para el gestor y 1 GB para el directorio temporal. Habrá que tener en cuenta adicionalmente el espacio para los ficheros que contienen los datos.

Memoria

Otro recurso fundamental en el buen rendimiento del sistema. Algunos SGBD son más apropiados para "servidores ligeros", mientras que otros parten con unos requisitos mínimos elevados.

Procesador y número de núcleos

Comprobaremos los requisitos mínimos del software en cuanto a procesador y tendremos en cuenta los núcleos que vamos a reservar, en caso de máquinas virtuales o contenedores.

1.7.4. Conectividad

Debemos tener en cuenta las necesidades de intercambio de información de la organización para asegurar una calidad mínima de la conexión. Esto es, asignar el ancho de banda apropiado, definir los parámetros de red (dns, ip, puerto, etc.). Asimismo, en caso de usar servicios cloud, debemos tener en cuenta la localización de los datos, tanto por legislación y costes asociados a la transferencia de datos entre localizaciones remotas y garantizar un máximo de latencia.



1.8.

Diferentes casos de uso

Con el siguiente diagrama podemos hacernos una idea sobre el sistema más apropiado según criterios de tamaño (volumen) o complejidad de los datos (variedad u horizontalidad)

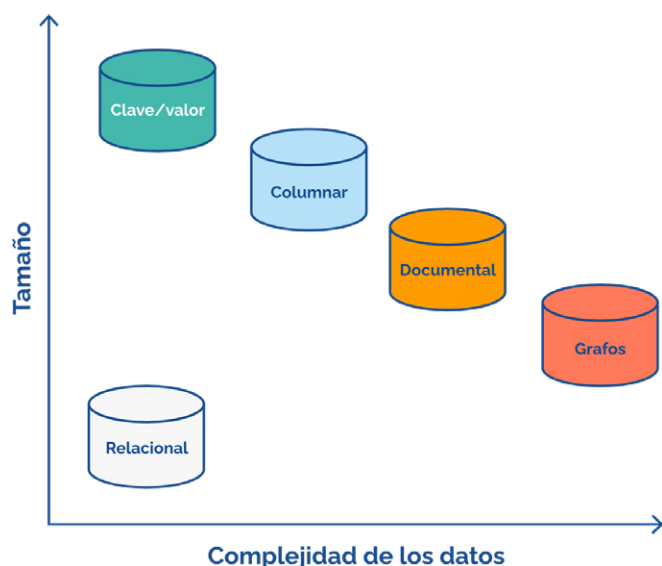


Imagen 3. Tipos del almacenamiento de datos en estructuras NoSQL

1.8.1. SGBD para aplicación móvil

Planteamos un escenario en el que se requiere la instalación de un micro SGBD local sobre un smartphone. SQLite encaja muy bien en estos casos.

1.8.2. Pyme

Nos referimos a una situación en la que vamos a tener un volumen pequeño/mediano de datos muy estructurados, pocas aplicaciones que accedan a los mismos (ERP/CRM).

No necesariamente se requiere conectividad externa, por lo que normalmente la empresa dispondrá de un servidor LAN. La elección dependerá de los requisitos de coste, sistema operativo del servidor, o dependencia del ERP.

1.8.3. Tienda Online

En este caso, tenemos pocas perspectivas de crecimiento horizontal de los datos (pocas tablas). Los SGBD Open Source encajan bien debido al buen soporte multiusuario. Es una buena situación en la que es recomendable optar por SaaS en la nube. Nos despreocuparemos de los problemas habituales de escalabilidad y altos costos de administración.

1.8.4. Operaciones de una organización grande

Se suele trabajar con grandes volúmenes de información, normalmente estructurada y con situaciones de alta concurrencia. El criterio económico en este caso es menos prioritario, y deberemos guiarnos por razones de compatibilidad y soporte. La velocidad de funcionamiento es fundamental.

1.8.5. Datawarehouse corporativo

Al contrario que en un SGBD orientado a operaciones, estas bases de datos no requieren una alta concurrencia, pero que mueven gran cantidad de datos, por lo que se requiere un gran ancho de banda, espacio en disco y memoria RAM. Aumenta la horizontalidad de los datos, por lo que no es imprescindible recurrir a un sistema relacional.

1.8.6. Data Lake

También es un almacén de datos como el datawarehouse, pero se almacenan en bruto de forma no estructurada. Nos encontraremos con diferentes tecnologías de almacenamiento y trabajaremos con frecuencia con las tecnologías de almacenamiento en la nube. Recomendaremos estar al corriente de los últimos productos o servicios que ofertan las grandes empresas como Amazon, Google o Microsoft. Son escenarios típicamente NoSQL y en entornos Big Data.

1.9.

Instalación de Oracle Database Express Edition

Oracle soporta varias plataformas. Vamos a realizar una instalación de la versión de sobre Windows. La última versión estable es de Octubre de 2021 *Oracle Database 21c Express Edition*.

[Descarga Oracle 21c XE](#)

1.9.1. Proceso de instalación

Una vez descargada la versión para Windows x64, lanzamos el asistente de instalación y continuaremos el proceso e indicaremos el directorio de instalación. Es recomendable que revise los requisitos mínimos de la instalación.

A continuación, introduciremos la contraseña del usuario administrador de la base de datos.

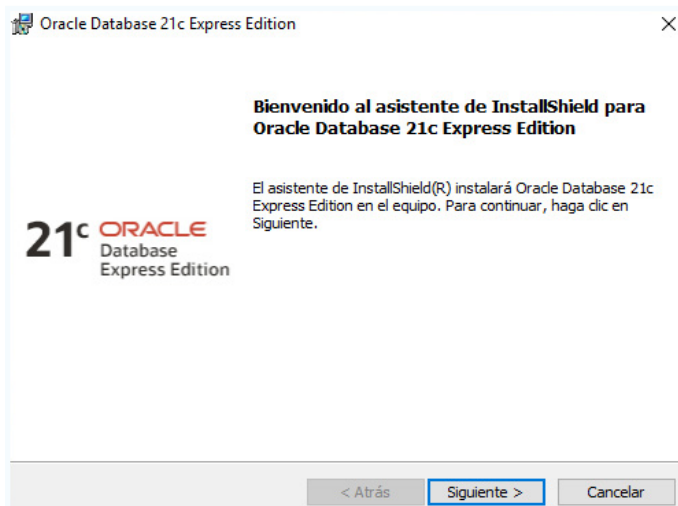


Imagen 4. Asistente de instalación de Oracle 21c XE x64.

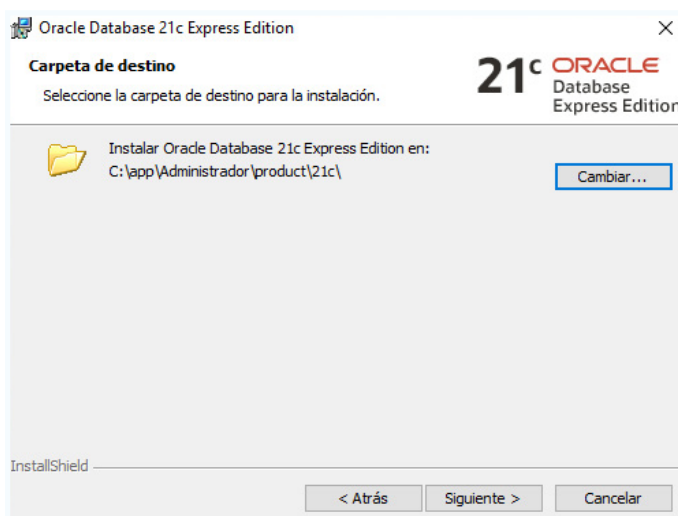


Imagen 5. Selección de directorio de la instalación.



Imagen 6. Contraseña de usuario administrador de la base de datos.

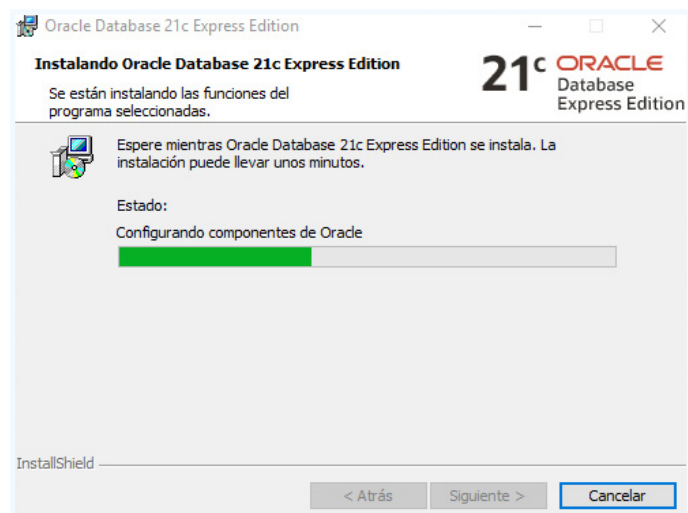


Imagen 7. Proceso de instalación.

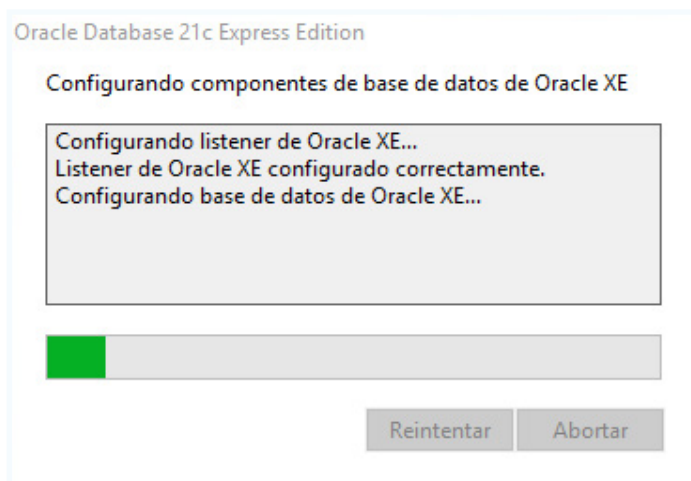


Imagen 8. Proceso de instalación, configuración inicial automática de la base de datos.

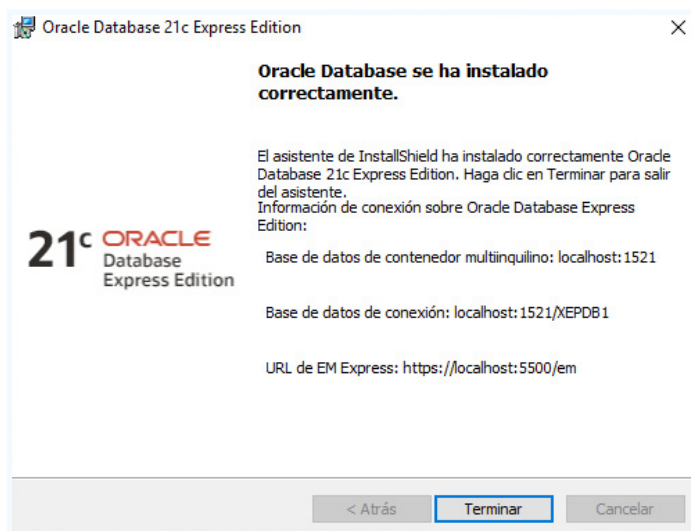


Imagen 6. Finalización del proceso de instalación.

1.9.2. Componentes instalados

El proceso de instalación de Oracle Database XE, instala varios componentes para facilitarnos las tareas de administración.

DBCA

Es un asistente que nos facilitará las tareas de creación, modificación y eliminación de nuestras bases de datos. Veamos un ejemplo de cómo utilizar este asistente para crear nuestra primera base de datos. Para ello, nos abriremos una ventana de comandos y ejecutaremos el comando *dbca*: El ejecutable se encuentra instalado en la ruta *C:\app\Administrador\product\21c\dbhomeXE\bin\dbca.bat*, aunque no es necesario situarse en ella, pues la variable de entorno del sistema se ha configurado correctamente.

La licencia del Oracle Express, solamente permite crear un SID. Vamos a recrear el proceso de creación, previamente, debemos suprimir la base de datos existente que se crea en el proceso de instalación.

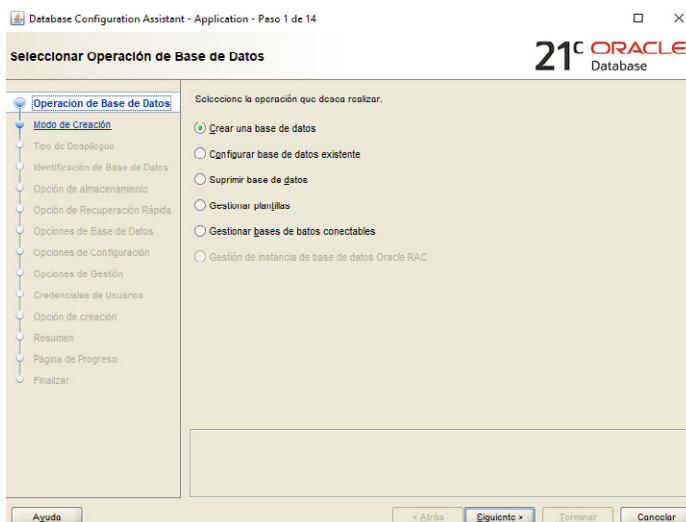


Imagen 7. Ventana de bienvenida de dbca

Tras pasar a la siguiente fase, haremos clic en configuración avanzada:

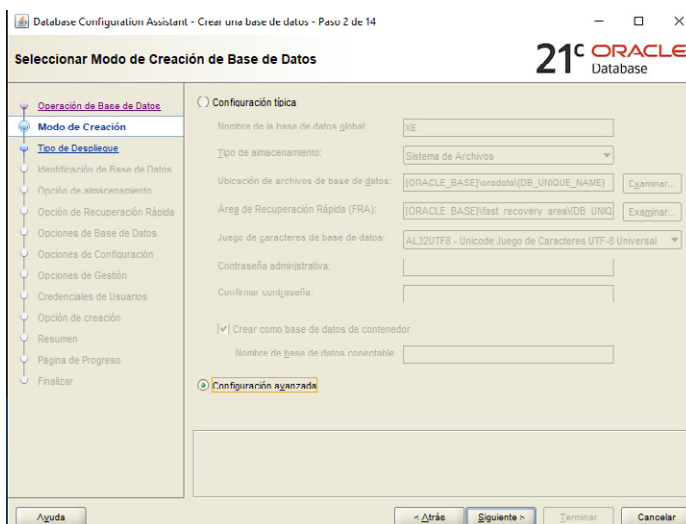


Imagen 8. Modo de creación avanzado

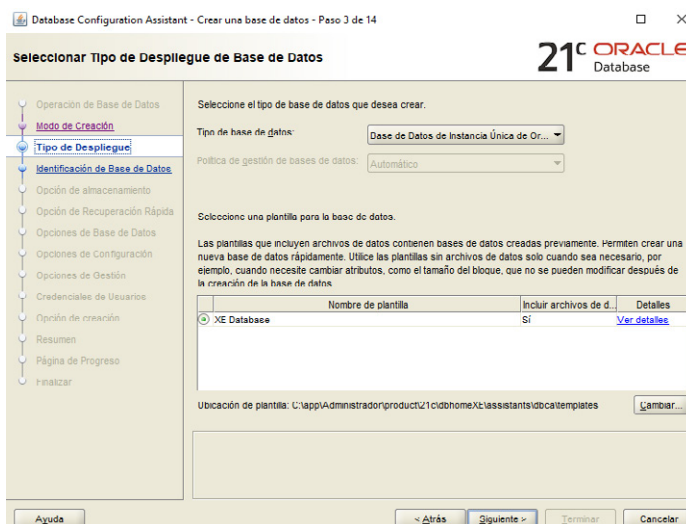


Imagen 9. Detalles de la configuración por defecto



Si en la ventana anterior hacemos clic en Ver detalles, podemos examinar la configuración por defecto de los parámetros de la base de datos. No te preocupes si no entiendes el significado de los parámetros. A lo largo del módulo los iremos viendo poco a poco, así como importancia de estos campos. Puedes guardar la configuración en formato HTML, para analizarla posteriormente.

Componentes de la Base de Datos	
Componente	Seleccionado
OMS	true
JSERVER	true
SPATIAL	true
MEDIA	true
ORACLE_TEXT	true
SAMPLE_SCHEMA	false
CWMLITE	true
JLV	true
NET_EXTENSIONS	true

Parámetros de Inicialización	
Nombre	Valor
db_name	(PROTOCOL=TCP) (SERVICE=(SID))DBD
dispatchers	(ORACLE_BASE)admin(DB_UNIQUE_NAME)adump
compatible	21.0.0
remote_login_passwordfile	EXCLUSIVE
undo_tablespace	UNDOTBS1
control_files	('ORACLE_BASE\oradata\DB_UNIQUE_NAME\control01.cfb', 'ORACLE_BASE\fast_recovery_area\DB_UNIQUE_NAME\control02.cfb')
diagnostic_dest	ORACLE_BASE
audit_trail	db
sh_block_size	8KB
open_cursors	300

Archivos de Datos		
Grupo	Tablespace	Tamaño
ORACLE_BASE\oradata\DB_UNIQUE_NAME\SYSAUX01.DBF	SYSAUX	460MB
ORACLE_BASE\oradata\DB_UNIQUE_NAME\SYSTEM01.DBF	SYSTEM	1110MB
ORACLE_BASE\oradata\DB_UNIQUE_NAME\UNDOTBS1.DBF	UNDOTBS1	25MB
ORACLE_BASE\oradata\DB_UNIQUE_NAME\USERS01.DBF	USERS	5MB

Archivos de Control	
Nombre	
ORACLE_BASE\oradata\DB_UNIQUE_NAME\control01.cfb	
ORACLE_BASE\fast_recovery_area\DB_UNIQUE_NAME\control02.cfb	

Grupos de Redo Logs	
Grupo	Tamaño
1	204800KB
2	204800KB
3	204800KB

Imagen 10. Parámetros por defecto

Pasamos a la siguiente ventana donde debemos indicar el nombre de la base de datos global y el nombre de la instancia (SID), a través de la cual nos conectaremos con ella. La instancia son los procesos y recursos que consume nuestra base de datos. Por defecto suele tener el mismo nombre de la primera base de datos.

Database Configuration Assistant - Crear una base de datos - Paso 4 de 14

Especificar Detalles de Identificación de Base de Da...

21c ORACLE Database

Operación de Base de Datos

Modo de Creación

Tipo de Despliegue

Identificación de Base de Datos

Opción de almacenamiento

Opción de Recuperación Rápida

Opciones de Base de Datos

Opciones de Configuración

Opciones de Gestión

Credenciales de Usuarios

Opción de creación

Resumen

Página de Progreso

Finalizar

Proporcione la información del identificador único de la base de datos. Una base de datos de Oracle se identifica de forma única mediante el nombre de la base de datos global, normalmente con el formato "nombre.dominio".

Nombre de la base de datos global: XE

SID: XE

Nombre de servicio:

☒ Crear como base de datos de contenedor

Una base de datos de contenedor se puede utilizar para consolidar varias bases de datos en una sola y permite la virtualización de bases de datos. Una base de datos de contenedor (CDB) puede tener cero o más bases de datos de conexión (PDB).

☒ Usar Deshacer tablespace local para PDB

☐ Crear base de datos de contenedor vacía

☒ Crear base de datos de contenedor con una o más PDB

Número de PDB: 1

Nombre de PDB: XEpdb1

Imagen 11. Configuración de nuestra primera BBDD

En este paso, comprobaremos qué es lo que ocurre al crear nuestra primera base de datos. Configuramos el nombre, establecemos el identificador de la misma, y crearemos el contenedor a la vez que la base de datos de conexión (PDB).

Vamos a avanzar dos pasos más, dejando los parámetros de almacenamiento y recuperación por defecto.

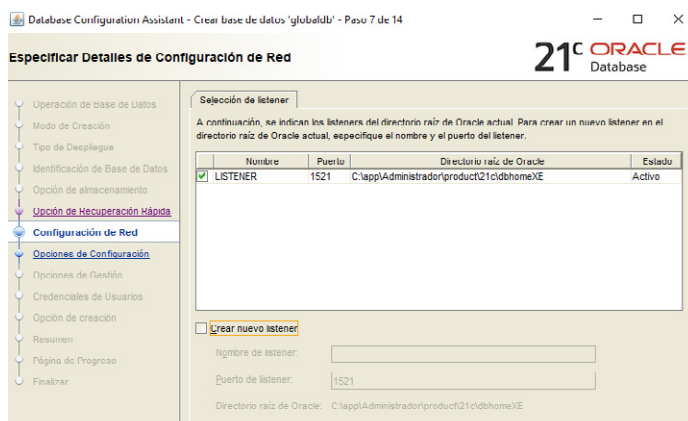


Imagen 12. Configuración de nuestra primera BBDD: Listener

Seguiremos avanzando, dejando los parámetros por defecto hasta llegar a la configuración de la memoria. Reservaremos una cantidad de memoria. Por ahora, no entraremos a fondo en la configuración de la memoria, pues es una tarea que requiere cierta experiencia. Vamos a marcar la gestión automática, reservando una cantidad fija de memoria, dependiendo de nuestros recursos disponibles. No existe una regla fija, como referencia podemos emplear entre un cuarto y un tercio de nuestra RAM para no saturar nuestro equipo.

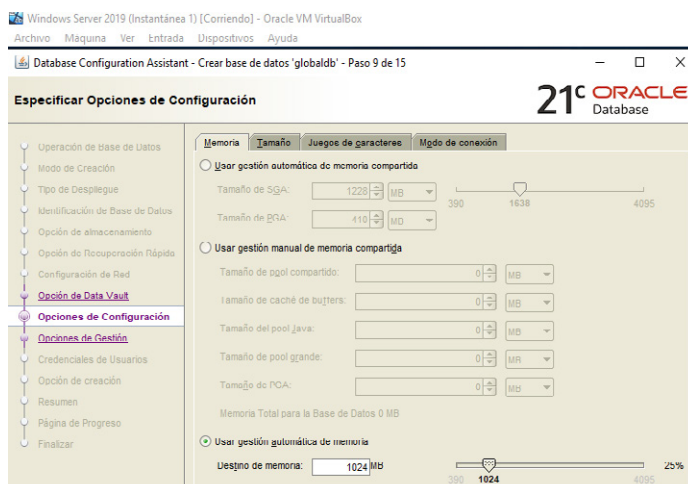


Imagen 13. Configuración de nuestra primera BBDD: Reserva de memoria

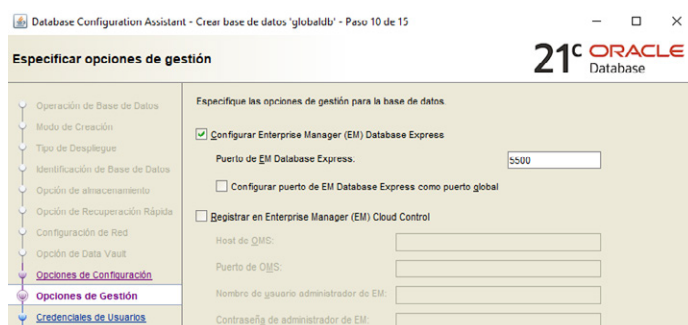


Imagen 14. Configuración de nuestra primera BBDD: Puerto EM DB Express

Como recordarás, ya configuraste previamente una única contraseña para los usuarios SYS, SYSTEM Y PDBADMIN. Por seguridad, para esta nueva base de datos, se recomienda especificar contraseñas diferentes.

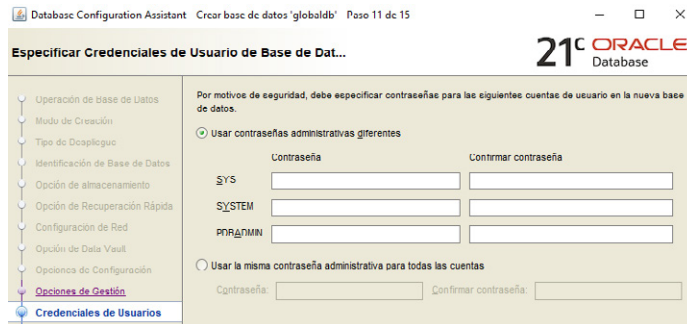


Imagen 15. Configuración de nuestra primera BBDD:
Contraseñas administrativas de la bdd nueva

Tras este paso, crearemos la base de datos. El asistente nos permite ejecutar scripts sql posteriores a la creación, guardar el proceso como plantilla y generar scripts de creación.

El resto de parámetros, los dejaremos por defecto por el momento.

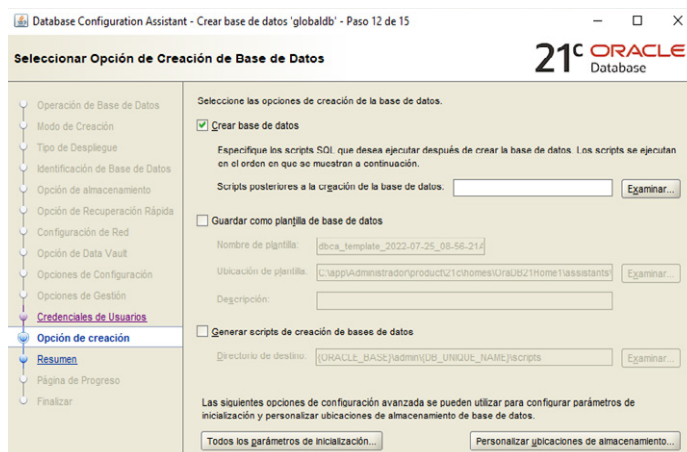


Imagen 16. Configuración de nuestra primera BBDD:
creación de la bdd nueva

Por último, marcaremos la opción terminar y esperaremos a que finalice el proceso:

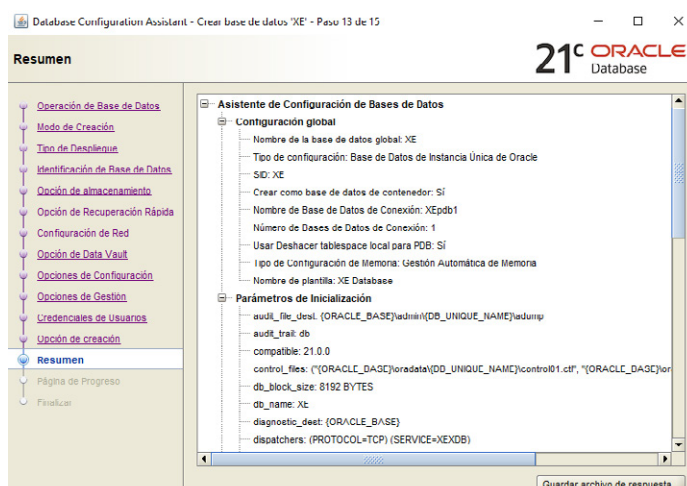


Imagen 17. Configuración de nuestra primera BBDD: último paso

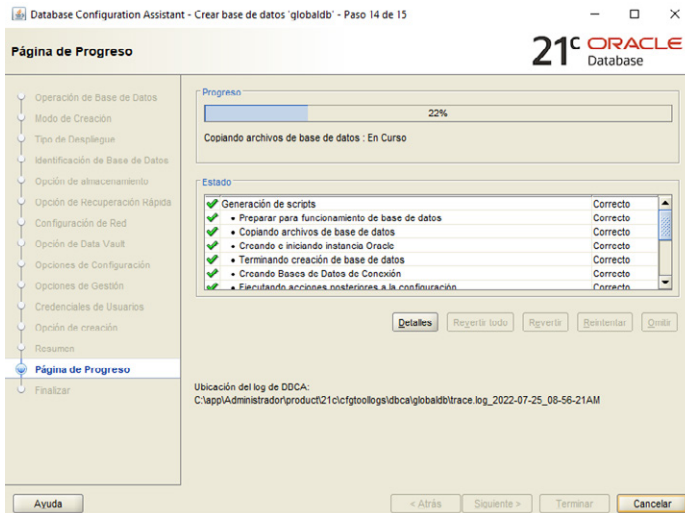


Imagen 18. Configuración de nuestra primera BBDD: último paso

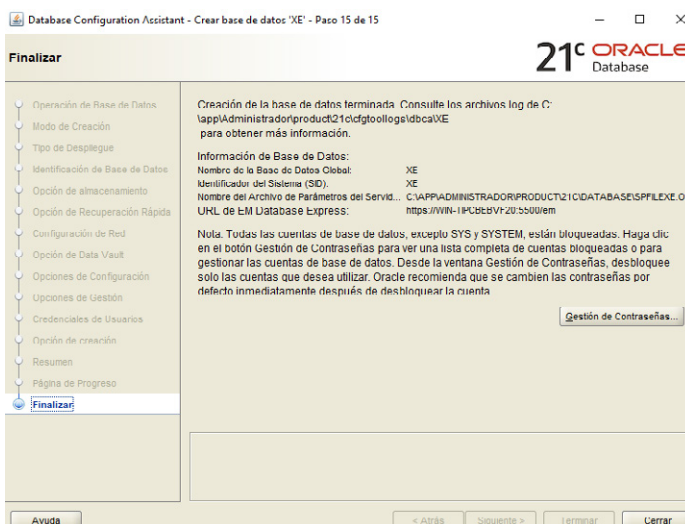
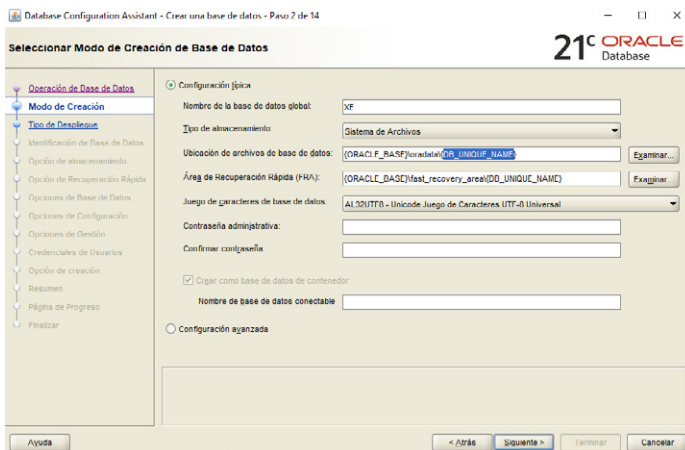


Imagen 19. Configuración de nuestra primera BBDD: Finalizado



Interfaces de usuario

Son aplicaciones cliente que nos van a facilitar la gestión y administración del servidor de base de datos. En el mercado hay varias herramientas con soporte a cualquier sistema gestor de base de datos. Si trabajamos con un sistema propietario como Oracle recomendamos sus herramientas:

- **SQL Developer:** Este software permite tanto la administración como el desarrollo de la estructura de datos. En las últimas versiones de Oracle, se debe instalar por separado. Su instalación es muy sencilla, basta con acudir a la [web oficial](#) y descargar la versión para nuestro sistema operativo:

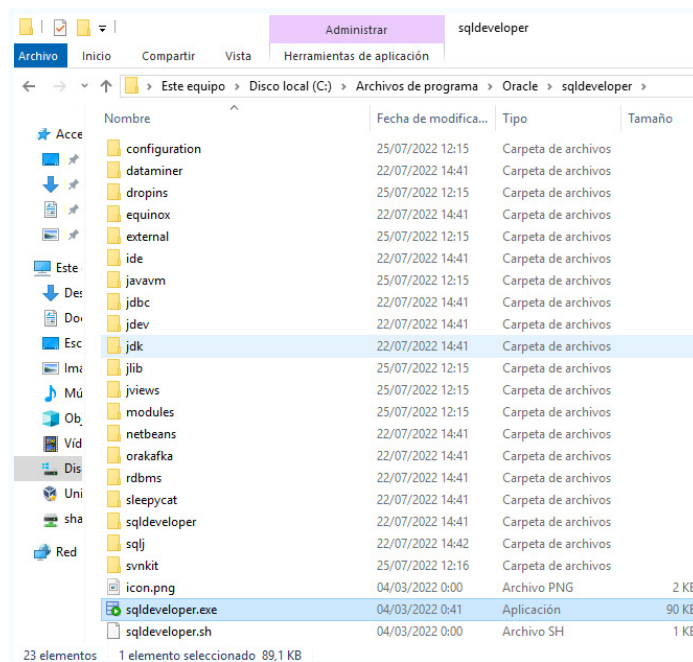
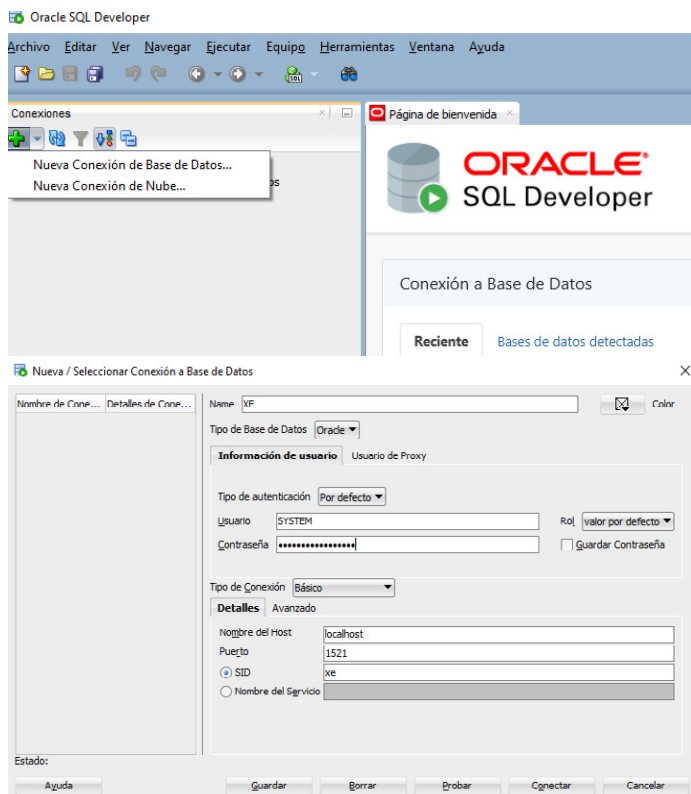


Imagen 20. Carpeta de Oracle SQL Developer

Copiaremos el contenido de dicha carpeta y ejecutaremos sqldeveloper.exe



Una vez nos aparezca la interfaz de usuario, procederemos a configurar nuestra conexión:



Una vez establecida la conexión, mediante este cliente, podremos realizar todo tipo de operaciones como crear tablas, usuarios, vistas, etc. Todo ello con la facilidad propia de una interfaz gráfica de usuario.

- > **SQL Plus:** Se conecta al servidor a través de una interfaz de comandos desde la cual podremos enviar comandos y sentencias SQL:
- > **Enterprise Manager (EM):** Se conecta al servidor mediante una interfaz gráfica de usuario que nos permitirá prácticamente la gestión completa de nuestra base de datos, incluyendo conexiones, monitorización, gestión de parámetros de configuración, usuarios, ficheros y espacio de almacenamiento, etc. Sin embargo, no nos sirve para la gestión de los datos. Existen dos versiones: Cloud Control y Express. La primera funciona en entorno web y es de pago, mientras que la segunda, se instala con el sistema gestor y está limitada a la monitorización del sistema.

Netc.A

Es el asistente que utilizaremos para manejar la configuración de los Listeners. Los listeners son los servicios responsables de escuchar peticiones de los usuarios e invocar el proceso de servidor.

RMAN

A través de este software, podremos gestionar las copias de seguridad de nuestra base de datos.

Netmgr

Con esta aplicación, gestionaremos todo lo relacionado con la conectividad a red de las aplicaciones y bases de datos.



 www.universae.com

